

Value Added Problems Problem 1:

Problem 1: Print all values in the BST within a given range

Approach:

Use DFS traversal (in-order) and prune branches using BST properties:

- If node value is greater than low, explore left subtree
- If node value is within range, include it
- If node value is less than high, explore right subtree

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {  
    int data;  
    struct Node* left;  
    struct Node* right;  
};
```

```
struct Node* createNode(int val) {  
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));  
    node->data = val;  
    node->left = node->right = NULL;  
    return node;  
}
```

```
void rangeQuery(struct Node* root, int low, int high) {  
    if (root == NULL) return;
```

```
if (root->data > low)
    rangeQuery(root->left, low, high);

if (root->data >= low && root->data <= high)
    printf("%d ", root->data);

if (root->data < high)
    rangeQuery(root->right, low, high);
}
```

```
int main() {
    printf("Name: Saumya Jade\n");
    printf("PRN: 25070521085\n");
    printf("Batch: B1\n");

    struct Node* root = createNode(10);
    root->left = createNode(5);
    root->right = createNode(15);
    root->left->left = createNode(3);
    root->left->right = createNode(7);
    root->right->right = createNode(18);

    int low = 7, high = 15;

    printf("Nodes in range: ");
    rangeQuery(root, low, high);

    printf("\n");
}
```

```
return 0;  
}
```

```
Name: Saumya Jade  
PRN: 25070521085  
Batch: B1  
Nodes in range: 7 10 15  
  
=== Code Execution Successful ===
```

Problem 2: Check if expression is balanced

Approach:

Use a stack:

- Push opening brackets
- On closing bracket, check top of stack
- If mismatch or empty → not balanced
- At end, stack must be empty

Problem 2: Balanced Brackets (C)

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX 1000000
```

```
char stack[MAX];
```

```
int top = -1;
```

```
void push(char c) {
```

```
    stack[++top] = c;
```

```
}
```

```
char pop() {
```

```
    return stack[top--];
```

```
}
```

```
int isEmpty() {
```

```
    return top == -1;
```

```
}
```

```
int isBalanced(char* s) {
```

```
    for (int i = 0; s[i] != '\0'; i++) {
```

```
        char c = s[i];
```

```
        if (c == '(' || c == '{' || c == '[')
```

```
            push(c);
```

```
        else {
```

```
            if (isEmpty()) return 0;
```

```
            char t = pop();
```

```

        if ((c == ')' && t != '(') ||
            (c == '}' && t != '{') ||
            (c == ']' && t != '['))
            return 0;
    }
}

return isEmpty();
}

int main() {
    char s[] = "{[()]}" ;

    printf("Name: Saumya Jade\n");
    printf("PRN: 25070521085\n");
    printf("Batch: B1\n");

    if (isBalanced(s))
        printf("Balanced\n");
    else
        printf("Not Balanced\n");

    return 0;
}

```

Output

Name: Saumya Jade

PRN: 25070521085

Batch: B1

Balanced

=== Code Execution Successful ===